Response Letter

We thank the reviewers for their valuable comments. The paper has been carefully revised according to the metareview, and all of the requested changes were implemented. Below is our detailed response to the metareview and each reviewer.

Replies to Metareview

Metareview Comment 1: Correct the description of SEDAR's comment-before-mutation and restore-after-mutation mechanism. Clearly explain the actual methodological differences between MORPH and SEDAR, avoiding misleading characterizations of the latter.
Reply: We have revised the description of SEDAR in Sections I (Introduction), II (Background and Motivation), and VI (Related Work) to accurately reflect its mutation-handling mechanism. Specifically, SEDAR temporarily comments out fragments incompatible with the downstream fuzzer's parser during mutation and restores them before executing the mutated SQL on the target DBMS. This mechanism does not permanently remove the original query logic. We have also revised the motivating example in Section II (Background and Motivation) to distinguish the intermediate representation used during mutation from the SQL executed by the target DBMS.

We have clarified the methodological differences between MORPH and SEDAR in Sections II (Background and Motivation) and VI (Related Work). SEDAR translates statements individually using their referenced source-side sub-schemas, while MORPH uses the AQT to propagate evolving target-side schema and type context across statements. In addition, MORPH uses parser and execution feedback to repair incompatibilities before mutation, whereas SEDAR's comment-and-restore mechanism primarily addresses compatibility with the downstream fuzzer during mutation.

We have also evaluated whether the transferred seeds preserve the behavior of the source test cases. On 100 randomly sampled MySQL-to-DuckDB test cases, MORPH achieves the highest result and state equivalence (52.0% and 74.0%), compared with 38.0% and 69.0% for SEDAR, as shown in the table below.

Source-to-target equivalence on 100 MySQL-to-DuckDB test cases.

Method	Result Eq. (%)	State Eq. (%)
SQLGlot	14.0	56.0
RISE	34.0	65.0
SEDAR	38.0	69.0
MORPH	**52.0**	**74.0**

Metareview Comment 2: Moderate the novelty claims and better distinguish MORPH from existing SQL translation approaches, particularly SEDAR, RISE, QTRAN, and CrackSQL. Explain more concretely what AQT and its associated mechanisms contribute beyond existing techniques, especially in the context of DBMS fuzzing.
Reply: We have revised Sections I (Introduction), II (Background and Motivation), III (Approach), and VI (Related Work) to clarify how MORPH relates to existing whole-test-case transfer and fuzzer-oriented seed adaptation approaches, in particular SEDAR. We have revised the related work in Section VI (Related Work) to state that MORPH adopts related techniques, including structural decomposition, localized translation, and dialect-aware retrieval, but targets fuzzing seed trans-

fer, and to distinguish MORPH from SEDAR, CrackSQL, RISE, and QTRAN in terms of goals, challenges, methods, and results.

Different goals. CrackSQL and RISE target query migration: they translate one self-contained query or procedural artifact at a time and aim to produce an equivalent target query. QTRAN targets oracle transfer: it translates a query and regenerates its target-side mutant so that the metamorphic relation is preserved for logic-bug detection. SEDAR and MORPH share the goal of transferring existing test cases into fuzzing seeds. For this goal, a transferred seed must execute on the target DBMS, retain testing-relevant SQL structures, and remain parsable and mutable by the downstream fuzzer.

Different challenges. A fuzzing seed is an ordered sequence of independently executed statements, and later statements may depend on the target-side state produced by earlier statements. For example, in `ALTER TABLE t ADD COLUMN b INT; SELECT b FROM t;`, the `SELECT` is valid only if the translated `ALTER TABLE` has produced the expected column on the target DBMS. When the target dialect changes a type, a constraint, or a DDL form, subsequent statements must follow the translated target state rather than the source state. Translating each statement correctly in isolation is therefore insufficient. Our focus is on maintaining this evolving target-side context across an ordered test sequence, beyond translating a self-contained query or preserving an oracle relation. SEDAR addresses the same setting but translates each statement using a source-side sub-schema and does not maintain translated target-side context across statements.

Different methods. AQT is built on a standard SQL AST; its contribution is not the tree representation itself but the cross-statement information it adds and how this information is consumed during transfer. First, AQT records forward read-after-write dependency edges between statements. These edges identify dependency-relevant preceding statements for context selection during instantiation and repair, while the original statement order is retained. Second, AQT records the cumulative schema context of the source test case, including object-state summaries such as object existence, table definitions, column names and types, constraints, and DDL-induced schema changes. During instantiation, MORPH maintains the corresponding target-side context and updates it after each translated statement. Each statement is thus instantiated and repaired against the translated target state. Third, abstract labels separate dialect-specific constructs from the surrounding structure, so that dialect-aware instantiation remains local while the original structure is preserved. Finally, MORPH uses parser and execution feedback to repair transferred seeds before mutation, whereas SEDAR temporarily comments out parser-incompatible fragments during mutation and restores them afterward. As described in their papers, the workflows of CrackSQL, RISE, and QTRAN do not propagate target-side context across the statements of a test case. We have updated Table I and Sections II (Background and Motivation) and VI (Related Work) to reflect these differences.

Different results. The evaluation in Section IV (Evaluation) provides evidence for the practical value of this design. Disabling cross-statement context propagation reduces execution validity from 0.716 to 0.523, supporting the contribution of target-side context propagation. Replacing dependency-based context selection with a recency-based window or the full context reduces execution validity to 0.599 and 0.643, respectively, while increasing input tokens (the reply to **Question** 1.2). At the system level, MORPH improves execution validity by 130% and branch coverage by 14% over SEDAR, and the controlled mutation-handling comparison is described in the reply to **Comment** 2.5.

Metareview Comment 3: Clarify how MORPH differs from and complements SmartFuzz in seed construction. The additional comparison is useful but limited in its experimental scope. The revision should acknowledge these limitations and avoid overstating comparative advantages.

Reply: We have clarified how MORPH differs from and complements SmartFuzz in Section VI (Related Work), and we have extended the comparison in Section IV (Evaluation). SmartFuzz uses LLMs to synthesize new seeds from features extracted from official DBMS documentation and historical bug-triggering inputs, whereas MORPH transfers existing test cases from mature DBMSs, which cover a broad range of SQL features, and preserves their cross-statement structures. The two approaches therefore provide complementary seeds: SmartFuzz targets bug-prone features, whereas MORPH provides broad feature coverage.

To compare them under the same setting, we use the seeds released by SmartFuzz and run them with GRIFFIN on all four target DBMSs, with the same DBMS versions, resource allocation, campaign duration, and coverage measurement as the other tools in RQ2. As shown in Table III, Table IV, and Figure 7 below, MORPH covers 422,130 branches and detects 22 bugs in total, compared with 394,825 branches and 17 bugs for SmartFuzz, i.e., 7% more branches and 5 more bugs. On Virtuoso, SmartFuzz achieves slightly higher coverage than MORPH, and both detect 3 bugs.

TABLE III: Number of branches covered by different tools (24h).

DBMS	SQLancer	SQLRight	Squirrel	GRIFFIN	SEDAR	SmartFuzz	MORPH
MonetDB	-	11,108	43,747	61,981	76,142	75,242	**81,486**
Virtuoso	-	9,207	26,302	28,017	36,583	**40,773**	40,183
DuckDB	22,188	25,624	96,434	96,512	114,876	130,966	**144,822**
ClickHouse	25,618	22,383	-	87,388	141,215	147,844	**155,639**
Total	47,806	68,322	166,483	273,898	368,816	394,825	422,130
Improvement*	528%↑	518%↑	60%↑	54%↑	14%↑	7%↑	-

* Calculated only for commonly supported DBMSs.

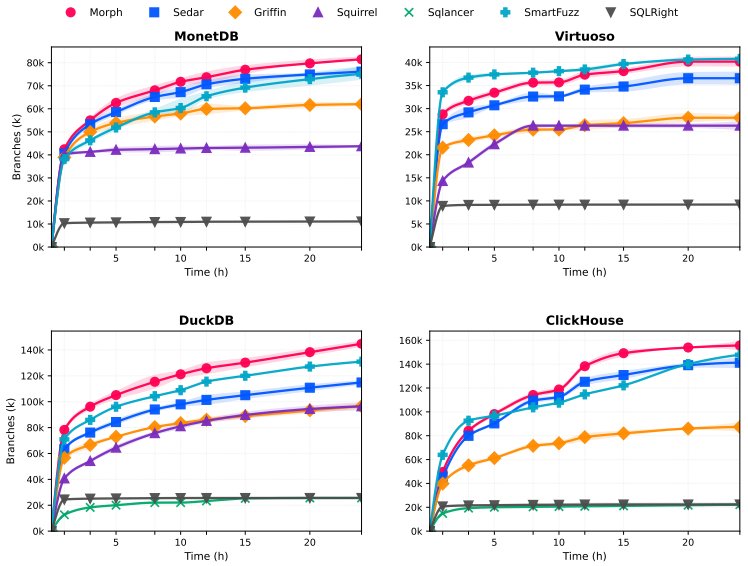


Figure 7: Coverage growth over 24-hour fuzzing campaigns. Displayed are the means and the 95% confidence intervals of five runs.

TABLE IV: Number of bugs detected by different tools (24h).

DBMS	SQLancer	SQLRight	Squirrel	GRIFFIN	SEDAR	SmartFuzz	MORPH
MonetDB	-	0	0	0	3	11	**14**
Virtuoso	-	0	0	0	2	**3**	**3**
DuckDB	0	0	0	0	1	2	**3**
ClickHouse	0	0	-	0	0	1	**2**
Total	0	0	0	0	6	17	22
Increment	22↑	22↑	22↑	22↑	16↑	5↑	-

To examine their complementarity, we further run GRIFFIN with the union of the SmartFuzz and MORPH seeds, which covers 413,021 branches in total (73,315, 48,030, 132,077, and 159,599 branches on MonetDB, Virtuoso, DuckDB, and ClickHouse, respectively) and detects 35 bugs in total, i.e., 13 and 18 more bugs than MORPH alone and SmartFuzz alone, respectively. The combination detects more bugs than either seed source alone on all four DBMSs, and achieves higher coverage on Virtuoso and ClickHouse. On MonetDB and DuckDB, its coverage is lower than that of MORPH alone, and it executes 38.5% and 22.4% fewer test cases than SmartFuzz alone within 24 hours, respectively. The additional seeds reduce the execution throughput, leaving fewer mutation opportunities for potentially more valuable seeds.

Metareview Comment 4: Clarify the provenance of the SEDAR binary used in the experiments, particularly given conflicting reports about its public availability. Provide sufficient information about the artifact and configuration to support reproducibility.
Reply: We have clarified the provenance of the SEDAR binary in Section IV (Evaluation), stating that the SEDAR binary used in our experiments was obtained from the authors of SEDAR. This binary runs the complete SEDAR pipeline, including schema capture, end-to-end LLM translation, and mutability refinement. To support reproducibility, we have included the SEDAR binary, its experimental configuration, and all SEDAR-generated seeds used in our experiments in our artifact.

Replies to Reviewer A

Question 1.1: Can the authors evaluate source-to-target result or state equivalence?
Reply: We have added a source-to-target result and state equivalence study in Section IV (Evaluation). We randomly sampled 100 MySQL test cases and evaluated SQLGlot, RISE, SEDAR, and MORPH on the same cases for transfer to DuckDB. We compared query results and the resulting database states, including schemas and table contents. Translation or execution failures count as non-equivalent, and both rates use all 100 cases as the denominator. MORPH achieves 52.0% result equivalence and 74.0% state equivalence, compared with 14.0%/56.0% for SQLGlot, 34.0%/65.0% for RISE, and 38.0%/69.0% for SEDAR. This study provides supplementary evidence for the sampled source-target pair, without claiming general semantic equivalence. We have also renamed "Semantic Correctness" to "Execution Validity" throughout the paper and clarified its definition in Section IV (Evaluation) to distinguish runtime executability from result or state equivalence.

Source-to-target equivalence on 100 MySQL-to-DuckDB test cases.

Method	Result Eq. (%)	State Eq. (%)
SQLGlot	14.0	56.0
RISE	34.0	65.0
SEDAR	38.0	69.0
MORPH	**52.0**	**74.0**

Question 1.2: What independent benefit does the dependency graph provide beyond context propagation?

Reply: Context propagation maintains the target-side state of the test case, while the dependency graph determines which preceding statements are provided as context for the current statement. For each statement, MORPH uses Δ to select its direct predecessors, i.e., the preceding statements that write objects read by the current statement, keeps only the most recent one for each set of shared objects, and provides their source and translated statements in the prompts for instantiation and repair. For example, in `ALTER TABLE t ADD COLUMN b INT; SELECT b FROM t;`, the dependency edge selects the `ALTER TABLE` statement as context when translating and repairing the `SELECT`, regardless of how many unrelated statements appear between them.

To isolate this benefit, we have added an ablation study in Section IV (Evaluation). We compare four context-selection modes under the same LLM, knowledge base, source corpus, repair budget, and statement order: (1) MORPH, which selects dependency predecessors through Δ; (2) MORPH$_{win}$, which replaces dependency-based selection with the five most recent successfully translated statements while keeping the same object summaries, so that it differs from MORPH only in whether the dependency graph is used; (3) MORPH$_{all}$, which provides all preceding successfully translated statements together with the complete source-side and target-side context; and (4) MORPH$_{w/o\text{-}ctx}$, which disables target-side context propagation. We measure execution validity at the statement level, the number of predecessor statements provided per statement, the proportion of provided predecessors that are dependency-relevant (i.e., connected to the current statement in Δ), and input tokens per statement.

TABLE VII: Contribution of the dependency graph to context selection across all source-target pairs.

Mode	Exec.	Preds/Stmt	Relevant Preds (%)	Input Tokens/Stmt
MORPH	**0.716**	0.47	100.0	509.6
MORPH$_{win}$	0.599	1.19	22.6	578.4
MORPH$_{all}$	0.643	27.95	4.2	1,842.3
MORPH$_{w/o\text{-}ctx}$	0.523	0.49	100.0	488.5

As shown in the table, MORPH achieves higher execution validity than MORPH$_{win}$ and MORPH$_{all}$ (0.716 vs. 0.599 and 0.643), while reducing input tokens per statement by 11.9% and 72.3%, respectively. In MORPH$_{win}$ and MORPH$_{all}$, only 22.6% and 4.2% of the provided predecessors are dependency-relevant. In addition, 15.4% of statements select dependency predecessors outside the five most recent statements, which a recency-based window cannot capture. The context of MORPH$_{all}$ grows with test-case length: on test cases with more than 20 statements, its input tokens per statement are 4.98$\times$ those of MORPH. These results show that the independent benefit of the dependency graph lies in selecting a compact, dependency-relevant context whose size does not grow with test-case length, while achieving higher execution validity than larger contexts.

Question 1.3: How were CrackSQL, RISE, and SEDAR adapted and configured for the four target DBMSs, and how did the authors ensure a fair comparison?
Reply: We have added the configuration of the translation baselines in Section IV (Evaluation), and the number of initial seeds used for each target DBMS in the experimental setup. Each baseline follows its original translation workflow. SQLGlot and jOOQ use their built-in dialect support; CrackSQL and RISE use the official SQL documentation of each target DBMS as dialect knowledge, while SEDAR requires no additional dialect knowledge. Since CrackSQL and RISE are designed to translate one query or artifact at a time, we translate the statements of each test case individually and reassemble them in their original order, without schema context. SEDAR translates complete test cases using the extracted source schemas as context. To ensure a fair comparison, all methods transfer the same collected test files and use the same number of initial seeds for each target DBMS (5,879, 4,551, 5,850, and 5,990 for MonetDB, Virtuoso, DuckDB, and ClickHouse, respectively), and no method receives additional manually written translation rules.

Comment 1.4: MORPH should therefore not be positioned as the first whole-test-case transfer framework or the first system to adapt transferred seeds to downstream fuzzers.
Reply: We have revised Sections I (Introduction), II (Background and Motivation), III (Approach), and VI (Related Work) to clarify how MORPH relates to existing whole-test-case transfer and fuzzer-oriented seed adaptation approaches, in particular SEDAR, and the methodological differences between them. Details can be found in the reply to **Metareview Comment** 1 and the reply to **Metareview Comment** 2.

Comment 1.5: These works show that structural decomposition, localized translation, and dialect-aware retrieval are not individually new. However, none of them models an independently executed stateful fuzzing test sequence, maintains target-side context across its statements, or integrates the result with coverage-guided seed mutation in the manner proposed by MORPH.
Reply: We have clarified in Section VI (Related Work) that MORPH adopts related techniques, including structural decomposition, localized translation, and dialect-aware retrieval, but targets fuzzing seed transfer. It focuses on maintaining evolving target-side context across an ordered test sequence and on producing seeds that remain parsable and mutable by downstream fuzzers. We have also revised Section VI (Related Work) to distinguish MORPH from CrackSQL, RISE, and QTRAN in terms of their goals. Details can be found in the reply to **Metareview Comment** 2.

Comment 1.6: The related-work discussion contains several details that should be corrected. In particular, the paper states that CrackSQL employs genetic algorithms to mine mutation patterns, but the cited work instead uses functionality-based query processing, cross-dialect syntax embedding with contrastive learning, and local-to-global translation. In addition, describing RISE as strictly processing a single SQL statement is too narrow because it also evaluates complete stored procedures containing control flow and multiple DML statements. A more accurate distinction is that RISE processes one self-contained query or procedural artifact at a time rather than a test file containing an ordered sequence of independently executed statements.
Reply: We have corrected the related work in Section VI (Related Work). CrackSQL is now described as combining functionality-based query processing, cross-dialect syntax embedding with contrastive learning, and local-to-global translation. RISE is now described as processing one self-contained query or procedural artifact at a time, rather than an ordered test sequence of inde-

pendently executed statements.

Comment 1.7: It is therefore unclear what (G_{tc}) contributes independently beyond recording the dependencies and preserving an order that is already present. The reported ablation removes context propagation rather than the graph itself, so it establishes the value of context but not an independent benefit from (G_{tc}). The paper should explain whether the graph affects retrieval, prompting, repair, or any other step not apparent from the algorithms.
Reply: We have revised Algorithm 2 and its description in Section III (Approach). We have removed the stable topological sort, since the original statement order already satisfies all forward read-after-write dependencies, and we have made the use of the dependency graph explicit: SELECTCONTEXT uses Δ to select dependency-relevant predecessors, whose source and translated statements are provided in the prompts for instantiation and repair. We have also added an ablation study in Section IV (Evaluation) that isolates the dependency graph from context propagation. Details can be found in the reply to **Question** 1.2.

Comment 1.8: The paper should also define the content and construction of the "object-state summaries" in (D) and clarify which forms of state are represented. The current formal definition explicitly supports forward RAW dependencies and cumulative schema/type context. Broader descriptions such as preserving all schema, data, and state dependencies should be narrowed unless additional dependency forms are implemented.
Reply: We have defined the object-state summaries in Section III (Approach). Each summary records whether a schema object exists, its table definition, column names and types, constraints, and the schema changes induced by preceding DDL statements, and the summaries are updated incrementally in statement order. In addition, we have revised the broader descriptions in Sections III (Approach) and IV (Evaluation), e.g., "propagates schema, type, and state context" to "propagates schema and type context", so that they are limited to forward read-after-write dependencies and cumulative schema and type context.

Comment 1.9: I suggest renaming this metric to "Execution Validity" or "Runtime Validity" and narrowing statements concerning equivalent syntax, semantic-intent preservation, and high-fidelity transfer. A sampled source-to-target behavioral comparison would provide useful supplementary evidence.
Reply: We have renamed "Semantic Correctness" to "Execution Validity" throughout the paper and clarified its definition in Section IV (Evaluation). We have also revised the description of instantiation in Section III (Approach), revising "to guarantee that the generated queries remain semantically consistent with the target dialect" to "to keep the generated queries valid in the target dialect". Section III (Approach) also states that constant substitution and minimal commenting may change the behavior of transferred seeds. In addition, we have added a sampled source-to-target result and state equivalence study as supplementary evidence. Details can be found in the reply to **Question** 1.1.

Comment 1.10: The paper should explain how each baseline was extended to these target dialects, including the dialect documents, knowledge bases, translation rules, LLMs, prompts, and manual effort used. It should also explain how a statement- or artifact-level translator was applied to a

complete test sequence and what schema context, if any, was provided.
Reply: We have added the configuration of the translation baselines in Section IV (Evaluation). Each baseline is configured with its original translation mechanism and the target-dialect knowledge it requires. Specifically, CrackSQL and RISE use the official SQL documentation of each target DBMS as dialect knowledge, while SEDAR requires no additional dialect knowledge. No baseline receives additional manually written translation rules, so no manual effort is spent on adapting the baselines to the target dialects. Since CrackSQL and RISE are designed to translate one query or artifact at a time, they translate the statements of each test case individually and reassemble them in their original order, without schema context. SEDAR translates complete test cases using the extracted source schemas as context. All methods use the same source test cases for each target DBMS. The prompts and configurations used by all baselines are included in our artifact. Details can be found in the reply to **Question** 1.3.

Comment 1.11: The authors should confirm whether their reproduction includes the restoration step. If the motivating example is an observed failure in the reproduced system rather than the general SEDAR algorithm, this distinction should be stated explicitly.
Reply: We have clarified in Section IV (Evaluation) that our experiments use the SEDAR binary obtained from the authors of SEDAR, which runs its complete pipeline, including the restoration step of mutability refinement. We have revised the descriptions of SEDAR in Sections I (Introduction), II (Background and Motivation), and IV (Evaluation) accordingly. We have also stated explicitly in Section II (Background and Motivation) that the motivating example is a failure observed when running SEDAR on a POSTGRESQL seed, rather than a general property of the SEDAR algorithm. Details can be found in the reply to **Metareview Comment** 1 and the reply to **Metareview Comment** 4.

Comment 1.12: The paper reports average coverage over five runs, but Tables III and VII provide no variance, confidence intervals, or statistical tests. Such information is important for stochastic fuzzing experiments. It is also unclear whether the reported bug counts are unions across the five runs or results from a particular run.
Reply: We have reported the variability of the coverage results in Section IV (Evaluation). The coverage in Table III is averaged over five independent runs, and Figure 7 now states that it displays the means and the 95% confidence intervals of the five runs. We have also clarified that the reported bug counts are the union of unique bugs across the five runs after stack-trace deduplication and manual verification. The improvements in Table VIII (Table VII in the original submission) are statistically significant ($p < 0.05$).

Comment 1.13: The full bug-finding campaign discovers 28 bugs, whereas the 24-hour comparison reports 22. The duration and computational budget of the full campaign should be reported. Similarly, Table VIII should specify the currency of the cost, the unit of time, and the total offline cost of processing all test files.
Reply: We have reported in Section IV (Evaluation) that the full campaign in RQ1 runs MORPH for seven days on each target DBMS, whereas RQ2 compares fuzzing tools in five independent 24-hour runs and reports the union of unique bugs across these runs. We have also added the units to the column headers of Table IX (Table VIII in the original submission), i.e., Cost/TS ($) and Time/TS (s), and we have reported in Section V (Discussion) that processing all test files with

Gemini 2.5-Flash-Lite costs $96 in total.

Comment 1.14: Finally, the component ablation is performed only for PostgreSQL-to-DuckDB transfer. Extending at least the most important ablation, particularly context propagation, to additional source-target pairs would strengthen the generality of the findings.

Reply: We have extended the entire component ablation in RQ4 from the PostgreSQL-to-DuckDB transfer to all source-target pairs used in RQ3, and have updated the corresponding figure and tables in Section IV (Evaluation). Across these pairs, MORPH$_{abs}$ improves syntactic correctness from 43.1% to 73.6%, MORPH$_{rag}$ improves execution validity from 35.2% to 65.5%, and MORPH$_{full}$ achieves the highest syntactic correctness (87.1%) and execution validity (71.6%) (Figure 8). In particular, across these pairs, disabling target-side context propagation reduces execution validity from 0.716 to 0.523 (Table VII). The same table also reports the dependency-graph ablation. Details can be found in the reply to **Question** 1.2.

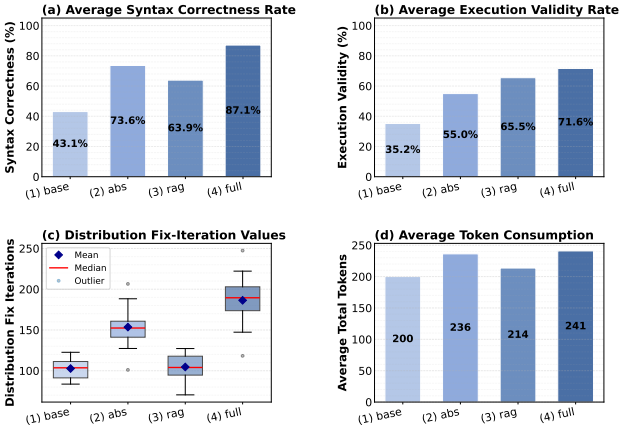


Figure 8: Ablation analysis of MORPH components across all source-target pairs.

Comment 1.15: The paper reports the target DBMS versions, corpus size, primary model, temperature, hardware environment, and an anonymous artifact.

For full reproducibility, the paper should additionally document the baseline adaptations, source-target test allocation, repeated-run aggregation procedure, full campaign duration, and validation procedure for automatically generated translation rules. Table IX reports that 136 rules required correction.

Reply: We have documented these details in the revised paper. (1) *Baseline adaptations:* Section IV (Evaluation) describes the dialect knowledge used by each baseline and how statement-level translators are applied to complete test cases (the reply to **Comment** 1.10). (2) *Source-target test allocation:* the experimental setup reports that all methods transfer the same collected test files and use the same number of initial seeds for each target DBMS, i.e., 5,879, 4,551, 5,850, and 5,990 seeds for MonetDB, Virtuoso, DuckDB, and ClickHouse, respectively. (3) *Repeated-run aggregation:* coverage is averaged over five independent runs, Figure 7 displays the means and the 95% confidence intervals, and bug counts are the union of unique bugs across the five runs after stack-trace deduplication and manual verification (the reply to **Comment** 1.12). (4) *Full campaign duration:* RQ1 runs MORPH for seven days on each target DBMS, whereas RQ2 consists of five independent 24-hour runs (the reply to **Comment** 1.13). (5) *Validation of generated rules:* Section V (Discussion) now describes that the generated rules are applied to translated test cases

and checked for errors, and that rules producing incorrect translations are manually inspected and corrected, which accounts for the 136 corrected rules in Table X.

Replies to Reviewer B

Question 2.1: Could the authors clarify whether the reported coverage/bug-finding improvements primarily stem from AQT-based translation or from a larger/richer effective seed corpus? In a controlled experiment where all baselines use exactly the same number and source of initial seeds, how much of the 14% coverage gain over SEDAR remains attributable to AQT?
Reply: We have added a seed-controlled comparison between MORPH and SEDAR in Section IV (Evaluation). Both methods transfer the same source test cases. For each target DBMS, we randomly sample 100, 200, and 300 seeds from the seeds generated by each method and use them as the initial seeds of GRIFFIN. Each configuration runs for 24 hours and is repeated five times, with the same DBMS version, GRIFFIN configuration, and resource allocation.

As shown in the table below, with the same number of initial seeds, MORPH achieves 47.3%–65.2% higher coverage than SEDAR on DuckDB, 17.2%–32.4% on MonetDB, 49.7%–75.0% on Virtuoso, and 57.8%–92.4% on ClickHouse. These gains are larger than the 14% gain with the full seed corpora, which shows that the coverage gain of MORPH over SEDAR is not due to a larger seed corpus. In addition, SEDAR detects no bugs in these settings, whereas MORPH detects one MonetDB bug with 200 and 300 seeds and one Virtuoso bug.

Branch coverage of SEDAR and MORPH with the same number of initial seeds.

DBMS	100 seeds		200 seeds		300 seeds	
	SEDAR	MORPH	SEDAR	MORPH	SEDAR	MORPH
DuckDB	49,745	**82,197**	58,938	**92,390**	68,499	**100,903**
MonetDB	20,406	**27,025**	28,173	**33,006**	29,948	**35,703**
Virtuoso	13,717	**24,002**	16,107	**26,670**	19,869	**29,745**
ClickHouse	35,967	**67,798**	41,478	**79,806**	55,852	**88,149**

Question 2.2: Regarding the characterization of SEDAR's comment-and-restore mechanism—if both MORPH and SEDAR ultimately feed compatible seeds to GRIFFIN, what is the exact difference in their post-mutation handling? Do seeds repaired by MORPH produce semantically equivalent mutated queries, whereas SEDAR's approach loses that equivalence? If not, what is the claimed advantage?
Reply: We have clarified the difference between the two mutation-handling strategies in Section IV (Evaluation). MORPH repairs parser-incompatible fragments before mutation, so the seeds fed to GRIFFIN are already compatible with both the fuzzer parser and the target DBMS, and no post-mutation handling is needed. SEDAR instead temporarily comments out parser-incompatible fragments during mutation and restores them afterward; restored fragments that contain translation errors therefore remain unrepaired and may cause the mutated test case to fail on the target DBMS, as illustrated by the motivating example in Section II (Background and Motivation). We have also stated explicitly that neither strategy guarantees semantic equivalence of the mutated queries. The advantage of MORPH is therefore not equivalence after mutation, but that it avoids carrying unrepaired translation errors into mutated test cases. Under controlled conditions with the same 300

translated seeds per target DBMS, MORPH's handling achieves 4.0%–27.2% higher branch coverage than SEDAR's handling. Before mutation, MORPH also achieves higher result and state equivalence than SEDAR on the sampled MySQL-to-DuckDB cases (52.0%/74.0% vs. 38.0%/69.0%). Details can be found in the reply to **Comment** 2.5 and the reply to **Question** 1.1.

Comment 2.3: Contribution of AQT and compositional baselines. The authors do not clearly articulate what fundamentally new capability AQT provides over existing SQL representations (e.g., JOOQ's parse tree or SQLGlot's AST) or translation systems (CrackSQL, RISE, QTRAN). More importantly, the evaluation lacks compositional baselines such as "existing SQL translator + SEDAR/GRIFFIN" or "AST-based translation + GRIFFIN".
Reply: We have clarified what the AQT contributes beyond existing SQL representations and translation systems in Sections II (Background and Motivation), III (Approach), and VI (Related Work). Details can be found in the reply to **Metareview Comment** 2.

We have also added compositional baselines on DuckDB in Section IV (Evaluation), which pair different translators with the same downstream fuzzer (GRIFFIN). All configurations transfer the same source test cases and run for 24 hours in five independent runs. MORPH covers 33.6%, 16.7%, and 26.1% more branches than SQLGlot+GRIFFIN, RISE+GRIFFIN, and SEDAR+GRIFFIN, respectively, and detects 3, 3, and 2 more bugs. On all four target DBMSs, MORPH also outperforms SEDAR+GRIFFIN, i.e., the SEDAR configuration in RQ2 (Table III and Table IV).

Comment 2.4: Incomplete discussion of closely related work. SmartFuzz (OOPSLA 2026) also studies high-quality seed generation for DBMS fuzzing and is directly relevant to the problem addressed by this paper. However, the paper neither discusses it sufficiently nor compares against it experimentally. The authors should add a discussion of SmartFuzz in Section VI (Related Work), clarify the differences in approach, and ideally include it as a baseline in the evaluation if feasible.
Reply: We have added a discussion of SmartFuzz in Section VI (Related Work), which clarifies that SmartFuzz synthesizes new seeds from DBMS documentation and historical bug-triggering inputs, whereas MORPH transfers existing test cases from mature DBMSs. We have also included SmartFuzz as a baseline in RQ2 and examined the complementarity of the two seed sources. Details can be found in the reply to **Metareview Comment** 3.

Comment 2.5: Potentially inaccurate characterization of SEDAR. The paper characterizes SEDAR's comment-and-restore mechanism as "destructive sanitization" that "sacrifices semantic depth." However, SEDAR's mechanism is designed to temporarily hide target-specific constructs from downstream fuzzers' strict parsers during mutation and restore them afterward. This is a pragmatic compatibility strategy, not a destructive removal of query logic.

The authors should revise their characterization to accurately reflect SEDAR's design intent and clarify the actual difference: MORPH performs pre-migration repair to make seeds parser-compatible, whereas SEDAR defers some compatibility handling until after mutation. A fairer comparison would evaluate both approaches under the same downstream fuzzer (e.g., GRIFFIN) and measure the net effect of each strategy on mutation effectiveness.
Reply: We have revised the characterization of SEDAR in Sections I (Introduction), II (Background and Motivation), and VI (Related Work) and removed the description of its comment-and-restore mechanism as destructive sanitization. We have also clarified the actual difference: MORPH repairs incompatibilities before mutation, whereas SEDAR defers compatibility handling until after

mutation.

We have added a controlled comparison in Section IV (Evaluation) to evaluate the two mutation-handling strategies. For each of the four target DBMSs, we applied both strategies to the same 300 translated seeds and ran GRIFFIN for 24 hours in five independent repetitions. Compared with SEDAR's strategy, MORPH's strategy achieves 27.2%, 15.5%, 4.0%, and 9.4% higher branch coverage on DuckDB, MonetDB, Virtuoso, and ClickHouse, respectively.

Comment 2.6: Questionable classification in Table I.
Reply: We acknowledge SQLGlot's multi-statement transpilation, jOOQ's schema metadata tracking, CrackSQL's DML support, and SEDAR's source-side handling of multi-statement dependencies. We have clarified the criteria for the three symbols in Table I in Section II (Background and Motivation) and revised the entries accordingly. SQLGlot's *Test Case Migration*, jOOQ's *Cross-stmt. Deps.*, and SEDAR's *Cross-stmt. Deps.* entries are now marked as △, while CrackSQL's *DML* entry is marked as ✓.

TABLE I: Capability comparison of existing SQL translation systems for test-case-level DBMS fuzzing seed transfer.

Tool	DDL	DML	DQL	Test Case Migration	Cross-stmt. Deps.
SQLGlot	✓	✓	✓	△	×
jOOQ	✓	✓	✓	×	△
RISE	△	△	✓	×	×
CrackSQL	△	✓	✓	×	×
QTRAN	△	△	✓	×	×
SEDAR	△	△	✓	✓	△
MORPH	✓	✓	✓	✓	✓

Cross-stmt. Deps. denotes explicit tracking of forward read-after-write dependencies and schema/type context across statements. ✓: supported and documented or evaluated; △: partially supported, i.e., supported only for some statements or settings, or through a separate component; ×: not supported.

Comment 2.7: Unclear source of the improvement. The reported coverage and bug-finding improvements may stem from a larger or more diverse effective seed corpus rather than the AQT-based translation technique itself. The authors should: (a) report the number of valid seeds generated by each method before fuzzing, (b) normalize the corpus size across baselines, or (c) conduct an experiment where seed counts are equalized.
Reply: We have conducted an experiment where seed counts are equalized in Section IV (Evaluation). MORPH and SEDAR transfer the same source test cases, and both use the same number of initial seeds (100, 200, and 300) for GRIFFIN. Under the same seed count, MORPH still achieves higher coverage than SEDAR. Details can be found in the reply to **Question** 2.1.

Comment 2.8: Translation overhead is unclear. Table VIII reports "Cost/TS" and "Time/TS" but does not specify the unit of measurement (e.g., USD per test suite? seconds per test suite?) or the size of the test suite being translated. Additionally, the paper does not compare MORPH's end-to-end translation time and LLM cost with previous methods such as SEDAR.

The authors should: (a) clearly define the units for cost and time, (b) report the average number of statements per test suite, (c) report per-seed translation time and cost, and (d) compare these overhead metrics against SEDAR and other baselines to demonstrate the practical feasibility of MORPH in real-world deployment.
Reply: We have clarified the translation overhead of MORPH in Section V (Discussion). (a) We have added the units to the column headers of Table IX (Table VIII in the original submission),

i.e., Cost/TS ($) and Time/TS (s); details can be found in the reply to **Comment** 1.13. (b) We have reported that each test suite contains 65.1 statements on average. (c) Across the four target DBMSs, MORPH takes 219 seconds and $0.004 per seed on average, and processing all test files costs $96 in total. (d) Under the same setting, SEDAR takes 146 seconds and $0.006 per seed, with a total cost of $144. Thus, MORPH costs less than SEDAR but takes longer per seed, since it performs additional instantiation and repair iterations. Because seed transfer is a one-time offline step and the resulting seeds can be reused across fuzzing campaigns, this overhead is acceptable in practice.

TABLE IX: LLM comparison. Syn.: syntactic correctness; Exec.: execution validity; TS: Test Suite.

Model	Syn.	Exec.	Cost/TS ($)	Time/TS (s)
Gemini 2.5-Flash-Lite	0.929	0.776	0.003	146.80
GPT-3.5-Turbo	0.890	0.533	0.015	208.66
DeepSeek-V3	0.986	0.699	0.003	231.11
Qwen3.5-27B*	0.971	0.852	0.119	1395.6

* Qwen3.5-27B with deep thinking mode.

Comment 2.9: Bug evidence is insufficient. The bug links are inaccessible, and the triggering test cases are not publicly available. The artifact includes complete and executable source code.
Reply: We have added the triggering test cases of all detected bugs to our anonymous artifact for bug verification and reproduction. The links to the bug reports will be released after acceptance to preserve double-blind review.

Replies to Reviewer C

Comment 3.1: The "semantic correctness" metric means only "executes without runtime errors," not result equivalence; a seed that runs but computes something different still counts. Using "+130-450%" as a headline overstates what was measured.
Reply: We have renamed "Semantic Correctness" to "Execution Validity" throughout the paper, including the abstract, the introduction, and Table V. The headline improvements (e.g., 130% over SEDAR and 100%–450% over the baselines) are therefore reported as gains in execution validity rather than semantic correctness. We have also clarified in Section IV (Evaluation) that execution validity measures executability on the target DBMS and does not imply result equivalence, and we have added a sampled result and state equivalence study. Details can be found in the reply to **Question** 1.1.

Comment 3.2: RQ2 baselines are somewhat hard to believe: zero bugs in 24 hours for SQUIRREL, GRIFFIN, and SQLancer across all four DBMSs. The baseline configuration is undocumented, several cells are "-", for unsupported targets, and the "528% over SQLancer" headline is computed over only the two DBMSs that SQLancer supports.
Reply: We have documented the configuration of the RQ2 baselines in Section IV (Evaluation). For SQLANCER, SQUIRREL, and GRIFFIN, we use the default configurations, seed corpora, and command lines from their official repositories, and we have included them in our artifact. The zero-bug results reflect that previously reported bugs were fixed in the newer DBMS versions, with

no new bugs found. In addition, the 528% improvement over SQLANCER is computed only on DuckDB and ClickHouse, the two target DBMSs it supports, as noted in the footnote of Table III.

Comment 3.3: Numeric inconsistency: the text claims RISE transfer improved semantic correctness "from 0.314 to 0.776," but Table V's own row reads RISE 0.314 / MORPH 0.748. One of these is wrong.
Reply: We have revised the text in Section IV (Evaluation) to state that, in the SQLite-to-DuckDB setting, MORPH improves execution validity from 0.314 (RISE) to 0.748, consistent with Table V. We have also checked the other numbers reported in Section IV (Evaluation) against Table V.

TABLE V: Syntactic correctness and execution validity of transferred seeds.

Method	MonetDB		Virtuoso		DuckDB		ClickHouse	
	Syn.	Exec.	Syn.	Exec.	Syn.	Exec.	Syn.	Exec.
No translation	0.589	0.236	0.324	0.097	0.674	0.284	0.613	0.182
SQLGlot	-	-	-	-	0.738	0.186	0.665	0.193
jOOQ	-	-	-	-	0.652	0.136	0.554	0.175
CrackSQL	-	-	-	-	0.740	0.211	0.717	0.264
RISE	0.830	0.266	0.545	0.178	0.857	0.314	0.764	0.278
SEDAR	0.781	0.388	0.562	0.169	0.895	0.365	0.821	0.315
MORPH	**0.929**	**0.776**	**0.670**	**0.629**	**0.939**	**0.748**	**0.895**	**0.694**
vs. SEDAR	+19%	+100%	+19%	+272%	+5%	+105%	+9%	+120%

Syn./Exec.: syntactic correctness/execution validity; "-": unsupported dialects.

Comment 3.4: Missing related work: SQLRight (USENIX Sec 2022) absent from RQ2; sqllogictest is never discussed; classic seed-selection literature ignored.
Reply: We have added SQLRight as a baseline in RQ2. Since SQLRight officially supports only SQLite, PostgreSQL, and MySQL, we have adapted it to the four target DBMSs while keeping its AFL-based queue scheduling, AST-based mutator, and oracle framework, and we have included the adaptation code in our artifact. The results can be found in the reply to **Metareview Comment** 3.

We have also extended the related work in Section VI (Related Work) with SQLRight, sqllogictest, and seed-selection techniques. SQLRight performs coverage-guided DBMS fuzzing for logic bugs, whereas MORPH constructs initial seeds for DBMS fuzzers. sqllogictest validates query results across database engines with a shared test format, whereas MORPH transfers test cases into seeds for coverage-guided fuzzing. Seed-selection and scheduling techniques, such as corpus minimization and power schedules (e.g., AFLFast), choose or prioritize seeds within an existing corpus, whereas MORPH constructs the initial corpus for target DBMSs that lack one; the two directions are therefore orthogonal and can be combined.

Comment 3.5: What's novel here is the conjunction of approaches: AST abstraction + RAG + parser-feedback repair, which are established ideas assembled around the AQT.
Reply: We have clarified in Section VI (Related Work) that MORPH adopts these techniques but focuses on maintaining evolving target-side context across an ordered test sequence through the AQT. Specifically, the AQT records forward read-after-write dependencies and cumulative schema context, which MORPH uses to select dependency-relevant context and to propagate the translated target-side state across statements during instantiation and repair. We have also added an ablation study in Section IV (Evaluation) that isolates the contributions of the dependency graph and context propagation. Details can be found in the reply to **Metareview Comment** 2 and the reply to **Question** 1.2.

Comment 3.6: RQ1 (28 bugs, unspecified "full campaign") vs RQ2 (22 bugs in 24h) relationship and campaign duration are unclear.
Reply: The full campaign in RQ1 runs MORPH for seven days on each target DBMS, whereas RQ2 compares fuzzing tools in five independent 24-hour runs and reports the union of unique bugs across these runs. Details can be found in the reply to **Comment** 1.13.

Comment 3.7: I would recommed tightening the validity claims: add a small differential check on transferred seeds (do they return equivalent results on a shared dialect subset?) to back the "semantic correctness" framing, and lean on coverage/bug counts rather than feature-richness as the primary evidence. For RQ2, publish baseline command lines and seed corpora in the artifact; the all-zero bug rows need more transparency, and SQLRight belongs in the comparison.
Reply: We have revised the paper following these suggestions. (1) *Differential check:* we have renamed "Semantic Correctness" to "Execution Validity" and added a sampled source-to-target result and state equivalence study (the reply to **Question** 1.1). (2) *Primary evidence:* our main claims rely on branch coverage and bug counts in RQ1 and RQ2, and the feature richness in RQ3 is presented as supplementary evidence of the testing value of transferred seeds. (3) *Baseline transparency:* we have documented the configurations, seed corpora, and command lines of the RQ2 baselines, included them in our artifact, and explained the zero-bug results (the reply to **Comment** 3.2). (4) *SQLRight:* we have added SQLRight to the RQ2 comparison (the reply to **Comment** 3.4).